HACKATHON · FRONTEND UI · MEMBER 3

Subscription Management System

Build one system to run plans, subscriptions and recurring billing in one place.

A unified platform for the full life of a subscription business: products and plans, quotations to subscriptions, automatic billing-day invoicing, discounts, payments and live revenue reporting — running on deterministic mock data today and swapping to the FastAPI + PostgreSQL API with a one-file change.

DOCUMENT **Phase 1 — Blueprint**	OWNER **Member 3 — Frontend UI**
STACK **React 18 · Vite · TypeScript · Tailwind · shadcn/ui**	DATA **2,000 subscriptions · 6,000 invoices**
CURRENCY **INR (₹) · Intl.NumberFormat("en-IN")**	STATUS **Ready for Phase 2 — Shell & Layout**

Contents — Dependencies · Five critical views · Finance dashboard · Wow feature · TypeScript contracts · Engine · Store · API surface · File tree · Edge cases · Demo story · Open questions

SECTION 01

Dependency plan

Run before Phase 2. Every package below is used; nothing speculative is installed.

```
# scaffold
npm create vite@latest frontend -- --template react-ts
cd frontend

# runtime
npm install react-router-dom date-fns next-themes recharts lucide-react clsx tailwind-merge \
  class-variance-authority tailwindcss-animate tw-animate-css @tanstack/react-table \
  react-window @types/react-window sonner

# tooling
npm install -D tailwindcss postcss autoprefixer @types/node
npx tailwindcss init -p
npx shadcn@latest init
```

```
npx shadcn@latest add button input label card table badge dialog sheet drawer popover \
  command tabs select slider switch checkbox textarea tooltip dropdown-menu separator avatar \
  skeleton alert alert-dialog calendar form progress scroll-area sonner chart breadcrumb \
  collapsible toggle-group
```

Choices worth defending. `sonner` supplies toasts (shadcn's `toast` is deprecated). `chart` is the shadcn Recharts wrapper, so all data-viz shares one theme. `react-window` is the fallback if TanStack pagination struggles with 6,000 invoices on the demo laptop — it is a swap, not a rewrite.

SECTION 02

The five critical views

Named in problem-statement vocabulary so the demo narration and the UI never drift apart.

#	VIEW	VOCABULARY	ROUTES	ROLE
1	**Auth & Role Shell**	Signup/Auth, Role (ADMIN / STAFF / CUSTOMER), password policy, Forgot-Password reset link	`/login /signup /forgot-password`	Public → all
2	**Admin Catalog & Rules Studio**	Products & Variants, Plans with Auto-Close / Early Close / Pause / Renew toggles, Tax Rules, Discount Rules, Quotation Templates, Add Staff	`/catalog/* /team`	ADMIN
3	**Staff Subscription & Quotation ERP**	Quotation Template → Create from Template, Subscription number, status chevron ribbon, order lines, Payment Terms, Renewal	`/subscriptions /subscriptions/:id /quotations/new`	ADMIN + STAFF
4	**Billing Day, Invoice & Payment**	Run Billing Day Batch, Simulate Billing Date, Invoice (snapshot lines, taxes, totals), Record Payment, Paid vs Outstanding, Overdue chase	`/billing /invoices /invoices/:id /payments`	ADMIN + STAFF

#	VIEW	VOCABULARY	ROUTES	ROLE
5	**Customer Portal & Finance Lead Dashboard**	Active Subscriptions, MRR Collected, Total Outstanding, Overdue Invoices, revenue collected, filters + summary view	`/dashboard /portal /portal/*`	Finance: ADMIN (full) + STAFF (read-only + chase) Portal: CUSTOMER

Route × role matrix

ROUTE	ADMIN	STAFF	CUSTOMER
`/login /signup /forgot-password`	•	•	•
`/dashboard — Finance Lead`	full	read-only + chase	—
`/catalog/products /catalog/plans /catalog/tax-rules`	•	—	—
`/catalog/discount-rules /catalog/quotation-templates`	•	—	—
`/team — Add Staff Member`	•	—	—
`/subscriptions /subscriptions/:id`	•	•	—
`/quotations/new`	•	•	—
`/billing`	•	•	—
`/invoices /invoices/:id /payments`	•	•	—
`/portal /portal/subscriptions/:id /portal/invoices/:id`	any	any	own only
`/`	→ /dashboard	→ /dashboard	→ /portal

`<RequireRole allow={['ADMIN','STAFF']}>` wraps every non-public route. A CUSTOMER deep-linking to `/invoices/12` gets a "Not your invoice" guard screen — not a 404.

Navigation labels

Revenue · Finance Lead Dashboard · Billing Day · Revenue Insights · Subscriptions · Subscriptions · Quotations · Renewals Due · Billing · Invoices · Payments · Overdue · Studio · Products & Variants · Plans · Tax Rules · Discount Rules · Quotation Templates · Team · Staff Members · Portal · My Subscriptions · My Invoices

SECTION 03

Finance Lead Dashboard

The 10-second test: how many subscriptions are active, how much came in, how much is owed, what is overdue.

KPI cards

Each card carries current value, target, `% change` with a directional arrow, a sparkline, an `animate-in fade-in slide-in-from-bottom-2` entrance and a count-up animation that respects `prefers-reduced-motion`.

CARD	FIELD	TARGET BASELINE	% CHANGE VS
Active Subscriptions	`active_subscriptions_count`	seeded target · 1,850	previous simulated day
MRR Collected	`mrr_collected`	quarter target · ₹48,00,000	previous billing cycle
Total Outstanding	`total_outstanding_owed`	≤ ₹5,00,000 healthy band	previous cycle
Overdue Invoices	`overdue_invoices_count`	0	previous cycle

> **QUERY TIME BADGE**
> Top-right of the header: **Query Time: 42 ms**. In mock mode it is measured with `performance.now()` around the selector; in live mode it reads `query_time_ms` from Contract C. Green < 200 ms, amber < 800 ms, red ≥ 800 ms — this makes the 2-second SLA visible to judges instead of merely claimed.

Overdue chase table

Columns — Invoice #, Customer, Amount Due, Due Date, Days Overdue (severity pill), Status, and actions *View* / *Send Reminder* / *Record Payment*.
Filters — due-date range, status (All / Overdue / Partially Paid / Confirmed), debounced customer search (250 ms), sort by days overdue. Pagination 25 / 50 / 100 through TanStack Table.

Charts (Recharts via shadcn chart)

1. **MRR vs Outstanding** — dual-series area across 12 simulated periods, live-updates after a billing run.
2. **Revenue Collected vs Outstanding** — stacked bar by Billing Frequency.
3. **Invoice status mix** — donut: Draft / Confirmed / Partially Paid / Paid / Cancelled.
4. **Renewals due next 30 days** — compact horizontal bar.

Interactive control panels

- **What-if Revenue Calculator** — plan select, billing frequency, quantity slider (1–200), discount-rule toggle, tax-rule select; recomputes through `calculateLineItem` and instantly updates the projected MRR vs outstanding chart.
- **Run Revenue Insights** (mock AI, 1.5 s spinner) — three urgency-bordered alerts: green (a discount rule auto-applied to upcoming invoices), yellow (renewals expiring within 7 days), red (overdue invoices at risk). Each has a working *Apply*.
- **Billing Day simulator shortcut** — deep-links to `/billing` with the clock pre-advanced.

SECTION 04

The standout “wow” feature

Billing Day Time-Travel Simulator — kept from the default proposal, sharpened so it proves the two hardest invariants.

> **WHY IT BEATS THE ALTERNATIVES**
>
> It *is* the core feature, its effect is measurable live in front of judges, and it proves idempotency and snapshot immutability rather than describing them. An AI copilot panel would demo conversation, not correctness.

What it does

1. **Simulate Billing Date** picker. `today` lives in the global store, so every page's overdue logic and `is_overdue` recompute together.
2. **Run Billing Day Batch** animates a five-stage pipeline: *Due subscriptions found (n)* → *Discount check* → *Tax computation* → *Invoice created* → *Invoice sent*, each stage ticking with `animate-in fade-in slide-in-from-bottom-2`.
3. Live KPI deltas: MRR ↑, Outstanding ↑, Overdue ↓ after reminders.
4. **Press Run again** — every record returns SKIP with `billing_period_key already exists` and the toast reads *“0 invoices created · 37 already billed — nothing billed twice.”*
5. A **Run Ledger** panel lists every decision (BILL / SKIP / AUTO_CLOSE + reason), so the idempotency proof is auditable rather than a single toast.
6. **Time-travel +1 month** — renewals advance with real `addMonths` math (Jan 31 → Feb 28) and auto-close fires on `expiry_date` when `auto_close` is true.

Pipeline contract

Due subscriptions → Idempotency key check → Discount window + usage → Tax rules → Invoice snapshot → Send / Ledger

SECTION 05

TypeScript contracts

frontend/src/types/contracts.ts — field names verbatim from the shared contract. Enums are string-literal unions; no `any` anywhere.

```ts
export type Role = 'ADMIN' | 'STAFF' | 'CUSTOMER';
export type ProductType = 'SERVICE' | 'ADDON';
export type BillingFrequency = 'DAILY' | 'WEEKLY' | 'MONTHLY' | 'YEARLY';
export type DiscountType = 'PERCENT' | 'FIXED';
export type DiscountScope = 'PRODUCT' | 'SUBSCRIPTION';
export type SubscriptionStatus = 'DRAFT' | 'QUOTATION' | 'CONFIRMED' | 'ACTIVE'
  | 'PAUSED' | 'CLOSED';
export type InvoiceStatus = 'DRAFT' | 'CONFIRMED' | 'PARTIALLY_PAID' | 'PAID'
  | 'CANCELLED';
export type PaymentMethod = 'UPI' | 'BANK_TRANSFER' | 'CARD' | 'CASH';

/** YYYY-MM-DD */
export type DateStr = string;

export interface User {
  id: number; email: string; full_name: string; role: Role;
```

```
  reset_token: string | null; created_at: DateStr;
}
export interface Product {
  id: number; name: string; product_type: ProductType;
  sales_price: number; cost_price: number; notes: string | null; is_active: boolean;
}
export interface ProductVariant {
  id: number; product_id: number; attribute_name: string;
  attribute_value: string; price_extra: number;
}
export interface TaxRule { id: number; name: string; rate_percent: number; is_active: boolean }

export interface DiscountRule {
  id: number; name: string; discount_type: DiscountType; value: number;
  scope: DiscountScope; min_qty: number; min_purchase: number;
  valid_from: DateStr; valid_until: DateStr | null;
  max_uses: number | null; current_uses: number;
}
export interface Plan {
  id: number; name: string; billing_frequency: BillingFrequency;
  plan_price: number; min_qty: number;
  available_from: DateStr; available_until: DateStr | null;
  auto_close: boolean; can_close_early: boolean;
  can_pause: boolean; can_renew: boolean;
}
export interface QuotationTemplate { id: number; name: string; plan_id: number; validity_days: number }
export interface QuotationTemplateLine {
  id: number; template_id: number; product_id: number;
  variant_id: number | null; quantity: number;
}

export interface Subscription {
  id: number; sub_number: string; customer_id: number; plan_id: number;
  discount_id: number | null; status: SubscriptionStatus;
  start_date: DateStr; next_billing_date: DateStr; expiry_date: DateStr | null;
  paused_at: DateStr | null; payment_terms_days: number;
}
export interface SubscriptionLine {
  id: number; subscription_id: number; product_id: number;
  variant_id: number | null; tax_id: number;
  discount_id: number | null; quantity: number; unit_price: number;
}

export interface Invoice {
  id: number; invoice_number: string; subscription_id: number;
  customer_id: number; billing_period_key: string; status: InvoiceStatus;
  issue_date: DateStr; due_date: DateStr; subtotal: number; total_discount: number;
  total_tax: number; grand_total: number; amount_paid: number;
}
export interface InvoiceLine {
  id: number; invoice_id: number; product_name_snapshot: string;
  variant_name_snapshot: string | null; quantity: number;
  unit_price_snapshot: number; discount_amount: number;
  tax_percent_snapshot: number; tax_amount: number; line_total: number;
}
export interface Payment {
  id: number; invoice_id: number; payment_method: PaymentMethod;
  amount: number; payment_date: DateStr; note: string | null;
}
```

Contract A — GET /api/subscriptions/{id}

```
export interface SubscriptionDetailLine {
  product_id: number; product_name: string; variant_name: string | null;
  quantity: number; unit_price: number; discount_amount: number;
```

```
  discount_applied: boolean; tax_percent: number; tax_amount: number;
  line_total: number;
}
export interface SubscriptionSummary {
  subtotal: number; line_discount_total: number; subscription_discount: number;
  total_discount: number; total_tax: number; grand_total: number;
}
export interface PlanPermissions {
  auto_close: boolean; can_close_early: boolean;
  can_pause: boolean; can_renew: boolean;
}
export interface SubscriptionDetail {
  id: number; sub_number: string; customer_id: number;
  customer_name: string; plan_id: number; plan_name: string;
  billing_frequency: BillingFrequency; status: SubscriptionStatus;
  start_date: DateStr; next_billing_date: DateStr; expiry_date: DateStr | null;
  paused_at: DateStr | null; plan_permissions: PlanPermissions;
  lines: SubscriptionDetailLine[]; summary: SubscriptionSummary;
}
```

Contract B — GET /api/invoices/{id}

```
export interface InvoiceLineSnapshot {
  product_name_snapshot: string; variant_name_snapshot: string | null;
  quantity: number; unit_price_snapshot: number; discount_amount: number;
  tax_percent_snapshot: number; tax_amount: number; line_total: number;
}
export interface PaymentRecord {
  id: number; payment_method: PaymentMethod; amount: number;
  payment_date: DateStr; note: string | null;
}
export interface InvoiceDetail {
  id: number; invoice_number: string; subscription_id: number;
  sub_number: string; customer_name: string; billing_period_key: string;
  status: InvoiceStatus; issue_date: DateStr; due_date: DateStr; is_overdue: boolean;
  subtotal: number; total_discount: number; total_tax: number;
  grand_total: number; amount_paid: number; amount_due: number;
  lines: InvoiceLineSnapshot[]; payments: PaymentRecord[];
}
```

Contract C — GET /api/analytics/summary

```
export interface OverdueInvoiceRow {
  invoice_id: number; invoice_number: string; customer_name: string;
  amount_due: number; due_date: DateStr; days_overdue: number;
}
export interface AnalyticsSummary {
  query_time_ms: number; active_subscriptions_count: number;
  mrr_collected: number; total_outstanding_owed: number;
  overdue_invoices_count: number; overdue_invoices: OverdueInvoiceRow[];
}
```

List projections — derived from A/B, confirm with Member 4

```
export interface Page<T> { items: T[]; total: number; page: number; page_size: number }
export interface SubscriptionListItem {
  id: number; sub_number: string; customer_name: string; plan_name: string;
  billing_frequency: BillingFrequency; status: SubscriptionStatus;
  next_billing_date: DateStr; expiry_date: DateStr | null; grand_total: number;
}
export interface InvoiceListItem {
```

```
  id: number; invoice_number: string; sub_number: string;
  customer_name: string; status: InvoiceStatus; issue_date: DateStr; due_date: DateStr;
  is_overdue: boolean; grand_total: number; amount_paid: number;
  amount_due: number;
}
export interface PaymentListItem extends PaymentRecord {
  invoice_number: string; customer_name: string;
}
```

SECTION 06

Mirrored business engine

frontend/src/mocks/engine/ — pure, framework-free, no React imports. Mirrors Member 2's Python math exactly so mock mode behaves like the real backend.

Engine types

```
export interface DiscountInput {
  type: DiscountType; value: number; min_qty: number; min_purchase: number;
  uses_left: number; valid_from: DateStr; valid_until: DateStr | null;
}
export interface LineItemResult {
  unit_price: number; gross_amount: number; discount_amount: number;
  discount_applied: boolean; taxable_amount: number;
  tax_amount: number; line_total: number;
}
export interface PlanFlags {
  auto_close: boolean; can_close_early: boolean;
  can_pause: boolean; can_renew: boolean;
}
export interface TransitionResult {
  allowed: boolean; next_status: SubscriptionStatus;
  next_billing_date: DateStr; expiry_date: DateStr | null;
  paused_at: DateStr | null; error: string | null;
}
export interface BillingDecision {
  action: 'BILL' | 'AUTO_CLOSE' | 'SKIP';
  billing_period_key: string; next_billing_date: DateStr;
  new_status: SubscriptionStatus; reason: string;
}
```

Three required pure functions

```
// engine/lineItem.ts
export function calculateLineItem(
  base_price: number, variant_extra: number, quantity: number,
  tax_percent: number, discount: DiscountInput | null, today_str: DateStr,
): LineItemResult;

// engine/transitions.ts
export function checkSubscriptionTransition(
  current_status: SubscriptionStatus, target_status: SubscriptionStatus,
  plan_flags: PlanFlags, next_billing_date: DateStr, expiry_date: DateStr | null,
  paused_at: DateStr | null, today_str: DateStr,
): TransitionResult;

// engine/billing.ts
export function evaluateSubscriptionForBilling(
  sub_number: string, status: SubscriptionStatus, billing_frequency: BillingFrequency,
```

```
  next_billing_date: DateStr, expiry_date: DateStr | null, auto_close: boolean,
  today_str: DateStr, existing_period_keys: ReadonlySet<string>,
): BillingDecision;
```

Supporting pure modules

MODULE	RESPONSIBILITY
money.ts	`roundHalfUp2` (sign-aware, mirrors Python `Decimal.ROUND_HALF_UP`), `sumLines`, INR formatter re-export
dates.ts	`addCycle(date, freq)` via `addDays/addWeeks/addMonths/addYears`; `diffDays`; `isWithinWindow`
payments.ts	`applyPaymentToInvoice(invoice, payments)` → `{ status, amount_paid, amount_due, is_overdue }`; CONFIRMED → PARTIALLY_PAID → PAID; refuses overpayment
billingBatch.ts	`runBillingDayBatch(state, today_str)` → `{ state, report }`. Per-subscription try/catch, increments `current_uses` only on a created invoice, returns new slices
snapshots.ts	`buildInvoiceLines(...)` — the only place invoice lines are created; nothing reads live catalog afterwards

Math, stated once

Unit = Base + VariantExtra → Gross = Unit × Qty → Discount gate → Taxable = Gross − Discount → Tax = Taxable × rate / 100 → Line total = Taxable + Tax

Discount applies only when: today inside `[valid_from, valid_until]`, `qty ≥ min_qty`, `gross ≥ min_purchase`, and `uses_left > 0`. Rounding is half-up to 2 decimals at the line level only.

Billing batch report

```
export interface BillingRunReport {
  ran_at: string; today: DateStr; billed: number; skipped: number;
  auto_closed: number;
  failed: { sub_number: string; reason: string }[];
  decisions: BillingDecision[]; duration_ms: number;
}
```

SECTION 07

Mock store shape

React context + useReducer, seeded once with a deterministic PRNG (mulberry32(1337)) so every demo reset reproduces identically.

```
export interface AppState {
  session: { user_id: number | null; role: Role; is_authenticated: boolean };
  today: DateStr;                       // the Time-Travel clock
  users: User[];
  products: Product[];
  product_variants: ProductVariant[];
  tax_rules: TaxRule[];
```

```
  discount_rules: DiscountRule[];
  plans: Plan[];
  quotation_templates: QuotationTemplate[];
  quotation_template_lines: QuotationTemplateLine[];
  subscriptions: Subscription[];
  subscription_lines: SubscriptionLine[];
  invoices: Invoice[];
  invoice_lines: InvoiceLine[];
  payments: Payment[];
  ui: {
    notifications: AppNotification[];
    notifications_read: boolean;
    palette_open: boolean;
    last_billing_report: BillingRunReport | null;
    insights: RevenueInsight[];
  };
  meta: { seeded_at: string; last_query_ms: number; data_version: number };
}

export interface AppNotification {
  id: string; kind: 'OVERDUE' | 'RENEWAL' | 'PAYMENT' | 'DISCOUNT';
  title: string; body: string; created_at: string; read: boolean; href: string;
}
export interface RevenueInsight {
  id: string; severity: 'GREEN' | 'YELLOW' | 'RED';
  title: string; body: string; action_label: string;
  action: 'APPLY_DISCOUNT' | 'RENEW_SUBSCRIPTIONS' | 'SEND_REMINDERS';
  applied: boolean;
}
```

Action union — all typed, no any

`SET_TODAY` · `TIME_TRAVEL` · `LOGIN` · `SIGNUP` · `LOGOUT` · `SWITCH_ROLE` · `REQUEST_PASSWORD_RESET` · `RUN_BILLING_BATCH` · `RECORD_PAYMENT` · `CONFIRM_INVOICE` · `SEND_INVOICE` · `CANCEL_INVOICE` · `SEND_REMINDER` · `CREATE_PRODUCT` · `UPDATE_PRODUCT` · `ADD_VARIANT` · `CREATE_PLAN` · `UPDATE_PLAN` · `CREATE_TAX_RULE` · `CREATE_DISCOUNT_RULE` · `UPDATE_DISCOUNT_RULE` · `TOGGLE_DISCOUNT_RULE` · `CREATE_QUOTATION_TEMPLATE` · `CREATE_SUBSCRIPTION_FROM_TEMPLATE` · `TRANSITION_SUBSCRIPTION` · `TOGGLE_ADDON` · `ADD_STAFF` · `MARK_NOTIFICATIONS_READ` · `APPLY_INSIGHT` · `RESET_DEMO_DATA` · `SET_QUERY_MS`

Derived selectors

Selectors live in `mocks/selectors/` and are memoized. Index maps (`subscriptionsById`, `invoicesBySubscription`, `existingPeriodKeys`, `paymentsByInvoice`) are built once per `data_version` inside a `useMemo`. This is what keeps the 2,000 / 6,000 dataset inside the 2-second budget.

Seed volume

ENTITY	COUNT	MIX
Subscriptions	2,000	1,850 ACTIVE · 60 PAUSED · 40 CONFIRMED · 50 CLOSED
Invoices	6,000	1,900 PAID · 1,100 PARTIALLY_PAID · 2,400 CONFIRMED · 450 overdue · 150 CANCELLED
Customers	1,200	small shops on monthly plans → large companies on yearly contracts
Payments	3,600	UPI · BANK_TRANSFER · CARD · CASH

STORY FIXTURES

Brightloop Cloud ERP — SERVICE, sales ₹2,000, cost ₹500, variant Brand = Odoo (+₹560 → ₹2,560). **WhatsApp Automated Alerts** — ADDON ₹300/month. **Annual Enterprise Plan** — YEARLY, min qty 5, can_pause true, can_close_early false. **10% Off 10+ Seats** — PERCENT, min_qty 10, max_uses 50, valid until Dec 31. **GST 18%** — tax rule.

SECTION 08

services/api.ts surface

The only module the UI may import. One flag decides mock or live; components never change.

```
const USE_MOCK = (import.meta.env.VITE_USE_MOCK ?? 'true') === 'true';
// every function: USE_MOCK ? mockAdapter.fn(...) : liveAdapter.fn(...)
```

Contract reads

```
getSubscription(id: number): Promise<SubscriptionDetail>;                // A
listSubscriptions(q: SubscriptionQuery): Promise<Page<SubscriptionListItem>>;
getInvoice(id: number): Promise<InvoiceDetail>;                           // B
listInvoices(q: InvoiceQuery): Promise<Page<InvoiceListItem>>;
listPayments(q: PaymentQuery): Promise<Page<PaymentListItem>>;
getAnalyticsSummary(q: AnalyticsQuery): Promise<AnalyticsSummary>;        // C
```

Billing cycle

```
runBillingDay(today: DateStr): Promise<BillingRunReport>;
previewBillingDay(today: DateStr): Promise<BillingDecision[]>;
```

Money

```
recordPayment(input: RecordPaymentInput): Promise<InvoiceDetail>;
```

Lifecycle

```
transitionSubscription(input: TransitionSubscriptionInput): Promise<SubscriptionDetail>;
toggleAddon(input: ToggleAddonInput): Promise<SubscriptionDetail>;
createSubscriptionFromTemplate(input: CreateFromTemplateInput): Promise<SubscriptionDetail>;
listQuotationTemplates(): Promise<QuotationTemplate[]>;
```

Admin studio

```
listProducts(): Promise<Product[]>;
createProduct(input: CreateProductInput): Promise<Product>;
updateProduct(input: UpdateProductInput): Promise<Product>;
addVariant(input: AddVariantInput): Promise<ProductVariant>;
listPlans(): Promise<Plan[]>;
createPlan(input: CreatePlanInput): Promise<Plan>;
updatePlan(input: UpdatePlanInput): Promise<Plan>;
listTaxRules(): Promise<TaxRule[]>;
createTaxRule(input: CreateTaxRuleInput): Promise<TaxRule>;
```

```
listDiscountRules(): Promise<DiscountRule[]>;
createDiscountRule(input: CreateDiscountRuleInput): Promise<DiscountRule>;
updateDiscountRule(input: UpdateDiscountRuleInput): Promise<DiscountRule>;
listStaff(): Promise<User[]>;
addStaff(input: AddStaffInput): Promise<User>;
```

Invoice operations & auth

```
confirmInvoice(id: number): Promise<InvoiceDetail>;
sendInvoice(id: number): Promise<InvoiceDetail>;      // mocked send + toast
cancelInvoice(id: number): Promise<InvoiceDetail>;
sendReminder(invoiceId: number): Promise<{ sent: true; to: string }>;

login(input: LoginInput): Promise<SessionUser>;
signup(input: SignupInput): Promise<SessionUser>;
requestPasswordReset(email: string): Promise<{ sent: true }>;
```

Portal & demo controls

```
getPortalOverview(): Promise<{ subscriptions: SubscriptionListItem[]; invoices: InvoiceListItem[] }>;
resetDemoData(): Promise<void>;
getClock(): Promise<{ today: DateStr }>;
setClock(today: DateStr): Promise<{ today: DateStr }>;
```

Live branch. `liveAdapter` is a typed `fetch` stub reading `VITE_API_BASE_URL`, with a `request<T>(path, init): Promise<T>` helper and 1:1 path mapping (`/api/subscriptions/:id` , `/api/invoices/:id` , `/api/analytics/summary`). Every mock function awaits 120–260 ms of artificial latency so loading states are real and testable.

SECTION 09

File tree

Ownership respected: pages, components, mocks and contract types are mine; services/api is the shared seam with Member 4.

```
frontend/
├── index.html
├── vite.config.ts
├── tailwind.config.ts
├── tsconfig.json
├── .env.example                    # VITE_API_BASE_URL=  VITE_USE_MOCK=true
└── src/
    ├── main.tsx
    ├── App.tsx                     # providers + router
    ├── index.css                   # tailwind, tw-animate-css, print stylesheet
    ├── types/
    │   ├── contracts.ts            # §5 — A/B/C + entities
    │   ├── api.ts                  # query/input DTOs
    │   └── nav.ts                  # role → nav map
    ├── services/
    │   ├── api.ts                  # USE_MOCK router (only import surface for UI)
    │   ├── mock.adapter.ts         # api.ts → store/engine bridge
    │   └── live.adapter.ts         # typed fetch stub
    ├── mocks/
    │   ├── seed.ts                 # deterministic 2k subs / 6k invoices
    │   ├── fixtures.ts             # the story data from the brief
    │   ├── engine/
    │   │   ├── types.ts
```

```
│   │   ├── money.ts
│   │   ├── dates.ts
│   │   ├── lineItem.ts
│   │   ├── transitions.ts
│   │   ├── billing.ts
│   │   ├── billingBatch.ts
│   │   ├── payments.ts
│   │   └── snapshots.ts
│   ├── store/
│   │   ├── AppStore.tsx         # context + useReducer
│   │   ├── reducer.ts
│   │   └── actions.ts
│   └── selectors/
│       ├── indexes.ts           # Maps + period-key Set
│       ├── analytics.ts         # Contract C, times itself
│       └── lists.ts             # filtered/paged projections
├── lib/
│   ├── format.ts                # formatINR (en-IN), formatDate, formatPercent
│   ├── cn.ts
│   ├── useDebouncedValue.ts
│   ├── useCountUp.ts
│   └── constants.ts             # status colors, chevron order, role nav
├── components/
│   ├── ui/                      # shadcn generated
│   ├── layout/
│   │   ├── AppShell.tsx
│   │   ├── Sidebar.tsx          # collapsible + mobile Sheet
│   │   ├── Topbar.tsx
│   │   ├── CommandPalette.tsx   # Cmd+K
│   │   ├── NotificationsPopover.tsx
│   │   ├── RoleSwitcher.tsx
│   │   ├── ThemeToggle.tsx
│   │   └── UserMenu.tsx         # logout confirm
│   ├── common/
│   │   ├── PageHeader.tsx
│   │   ├── EmptyState.tsx       # "No data captured yet…"
│   │   ├── LoadingSkeleton.tsx
│   │   ├── ErrorState.tsx
│   │   ├── MoneyText.tsx
│   │   ├── StatusBadge.tsx
│   │   ├── QueryTimeBadge.tsx
│   │   ├── DataTable.tsx        # TanStack wrapper
│   │   └── ConfirmDialog.tsx
│   ├── finance/
│   │   ├── KpiCard.tsx
│   │   ├── MrrOutstandingChart.tsx
│   │   ├── RevenueMixChart.tsx
│   │   ├── InvoiceStatusDonut.tsx
│   │   ├── OverdueChaseTable.tsx
│   │   ├── WhatIfCalculator.tsx
│   │   └── RevenueInsightsPanel.tsx
│   ├── subscriptions/
│   │   ├── StatusChevronRibbon.tsx
│   │   ├── SubscriptionTable.tsx
│   │   ├── SubscriptionHeaderCard.tsx
│   │   ├── OrderLinesTable.tsx
│   │   ├── PlanActionBar.tsx    # pause/close/renew, flag-gated + tooltip
│   │   └── TemplatePicker.tsx
│   ├── billing/
│   │   ├── TimeTravelBar.tsx
│   │   ├── RunBillingDayPanel.tsx
│   │   ├── PipelineStages.tsx
│   │   ├── RunLedger.tsx
│   │   ├── InvoiceTable.tsx
│   │   ├── InvoiceDocument.tsx  # print target
│   │   └── RecordPaymentDialog.tsx
│   ├── catalog/
│   │   ├── ProductDialog.tsx
│   │   ├── VariantEditor.tsx
│   │   ├── PlanDialog.tsx       # 4 permission toggles
│   │   ├── TaxRuleDialog.tsx
│   │   ├── DiscountRuleDialog.tsx
│   │   ├── QuotationTemplateDialog.tsx
│   │   └── AddStaffDialog.tsx
│   ├── portal/
│   │   ├── PortalSubscriptionCard.tsx
│   │   └── PortalInvoiceList.tsx
```

```
│   └── auth/
│       ├── PasswordChecklist.tsx
│       ├── AuthLayout.tsx
│       └── ForgotPasswordDialog.tsx
└── pages/
    ├── auth/LoginPage.tsx  SignupPage.tsx
    ├── DashboardPage.tsx
    ├── BillingDayPage.tsx
    ├── SubscriptionsPage.tsx  SubscriptionDetailPage.tsx
    ├── NewQuotationPage.tsx
    ├── InvoicesPage.tsx  InvoiceDetailPage.tsx
    ├── PaymentsPage.tsx
    ├── catalog/ProductsPage.tsx  PlansPage.tsx  TaxRulesPage.tsx
    │           DiscountRulesPage.tsx  QuotationTemplatesPage.tsx
    ├── TeamPage.tsx
    ├── portal/PortalHomePage.tsx  PortalSubscriptionPage.tsx  PortalInvoicePage.tsx
    └── NotFoundPage.tsx  ForbiddenPage.tsx
```

SECTION 10

Judge-breaker edge cases

Each one is visible in the UI, not buried in code — that is the difference between a demo and a system.

EDGE CASE	WHERE THE JUDGE SEES IT	MECHANISM
Double-billing guard	`/billing` — run the batch twice	`billing_period_key` set → SKIP + "0 created · 37 already billed" toast; Run Ledger shows SKIP reasons
Snapshot immutability	`/invoices/:id` after editing a Product price in Studio	Invoice renders `*_snapshot` columns only; before/after price banner proves the invoice is unchanged
Discount expiry / usage exhaustion	`/catalog/discount-rules` + next billing run	Past `valid_until` or `current_uses ≥ max_uses` → `discount_applied: false`; row shows 48/50 then 50/50 — exhausted
Pause date shift	`/subscriptions/:id` — pause 10 days, then resume	`next_billing_date` and `expiry_date` shift by exactly 10 days; ribbon shows PAUSED with `paused_at`
Partial payments	`/invoices/:id` — pay ₹1,000 of ₹3,020	CONFIRMED → PARTIALLY_PAID → PAID, `amount_due` recomputed, KPI Outstanding drops
Plan-flag-disabled buttons	`/subscriptions/:id` on Annual Enterprise Plan (`can_close_early: false`)	Disabled + tooltip "This plan does not allow early closure" — enforced by `checkSubscriptionTransition`, not the button
One bad record doesn't block the batch	`/billing` failed strip	Per-subscription `try/catch` → `failed[]` rendered as amber rows while the rest succeed
Tax computed from rules	Invoice lines	`tax_percent_snapshot` from the Tax Rule (GST 18%), never hardcoded
Unique email / password policy	`/signup`	Live checklist (9+, upper, lower, special) + duplicate-email error before submit
No double charge on cancel	`/invoices/:id` — cancel	CANCELLED invoices excluded from outstanding/overdue and from period-key reuse

ENFORCEMENT PRINCIPLE

Business rules live in the reducer and the mirrored engine, never in the view. Buttons are disabled *because* the engine refused, so a judge cannot click through the UI into an illegal state.

SECTION 11

Three-minute demo story

One continuous narrative: billing day is due, renewals are about to lapse, overdue invoices threaten revenue, a discount auto-applies — and after the actions MRR rises while Overdue falls.

TIME	ACTION	WHAT JUDGES SEE
0:00–0:15	Open `/dashboard` as **ADMIN**	KPIs load, *Query Time: 38 ms*, 1,850 active, ₹48.2L MRR, ₹6.4L outstanding, 412 overdue
0:15–0:35	Click **Run Revenue Insights**	1.5 s spinner → red "412 overdue at risk", yellow "9 renewals within 7 days", green "10% Off 10+ Seats will auto-apply"
0:35–0:55	**Apply** green, then red	Discount usage 41 → 42; reminders sent; Outstanding and Overdue tick down with animated counters
0:55–1:15	What-if: Annual Enterprise, YEARLY, qty slider 5 → 12, discount ON, GST 18%	Projected MRR recalculates live: ₹2,560 × 12, 10% off, 18% tax
1:15–1:45	`/billing` → Time-Travel to billing day → **Run Billing Day Batch**	Five-stage pipeline animation, 37 invoices created, MRR ↑, Outstanding ↑, Run Ledger populated
1:45–2:00	**Run it again**	"0 invoices created · 37 already billed — nothing billed twice." Idempotency proven
2:00–2:20	Open a generated invoice → **Record Payment** ₹2,000 of ₹3,020	PARTIALLY_PAID, `amount_due` ₹1,020, dashboard Outstanding drops; second payment → PAID
2:20–2:35	Edit "Brightloop Cloud ERP" price in Studio → return to invoice	Invoice still shows the ₹2,560 snapshot — immutability proven
2:35–2:50	Subscription detail → try **Close Early** on Annual plan	Disabled with tooltip; then **Pause** → time-travel → resume → dates shift exactly
2:50–3:00	Switch role to **CUSTOMER**	Sidebar collapses to Portal only; sees only own 2 subscriptions and 3 invoices. Logout confirm → done

Why this order works

- Opens with the finance question, not with configuration — matches the primary user.
- Every subsequent click moves a number the judge has already seen.
- The second billing run converts a claim into a proof.
- Closes on role isolation, the requirement most likely to be faked elsewhere.

SECTION 12

Open questions for the team

Nothing here blocks Phase 2 — these are assumptions I will code against unless corrected.

#	TOPIC	MOCK ASSUMPTION	NEEDS
1	**mrr_collected** definition	Normalized monthly value of ACTIVE subscriptions whose latest invoice is PAID (DAILY×30, WEEKLY×4.345, MONTHLY×1, YEARLY÷12)	Yes/no from Member 2 so mock and engine agree
2	**Discount stacking**	Line discount + subscription discount additive, capped at gross per line	Member 2
3	**Tax inclusive vs exclusive**	Exclusive — `Tax = Taxable × rate / 100`, per engine spec	Member 2
4	**Source of "today"**	Mock uses the Time-Travel clock; live mode hides the simulator and uses server date	Member 1
5	**List endpoint shapes**	`Page<T>` + params `page`, `page_size`, `status`, `q`, `from`, `to`	Member 4 to confirm param names

Phase gates

PHASE	DELIVERABLE	EXIT TEST
2	Auth screens, role shell, collapsible sidebar, Cmd+K palette, notifications, theme	Every nav item resolves; Cmd+K runs a real action; mobile drawer works
3	Finance dashboard, what-if calculator, Revenue Insights, Catalog & Rules Studio, Staff subscription views	KPIs react to actions; engine math matches hand calculation
4	Billing Day Time-Travel Simulator, invoice print, Record Payment	Second run bills nothing; partial payment transitions correctly
5	Polish, skeletons, empty states, accessibility	Keyboard-only pass; contrast checked; no console errors
6	README, demo script, pre-presentation checklist	`npm run build` clean; mock-to-live swap verified with Member 4